

Termín odevzdání:	04.05.2026 11:59:59
Pozdní odevzdání s penalizací:	15.05.2026 23:59:59 (Penále za pozdní odevzdání: 100.0000 %)
Hodnocení:	3.9600
Max. hodnocení:	3.0000 (bez bonusů)
Odevzdaná řešení:	2 / 20 Volné pokusy + 20 Penalizované pokusy (-2 % penalizace za každé odevzdání)
Nápovědy:	0 / 2 Volné nápovědy + 2 Penalizované nápovědy (-10 % penalizace za každou nápovědu)

Úkolem je realizovat sadu tříd, které budou simulovat datové struktury použité v DNS serveru.

DNS je síťová služba, která slouží k překladu adres z textové do binární podoby. Například adresa `progtest.fit.cvut.cz` je v současnosti přeložena na adresu `147.32.232.142`. Tento překlad je realizován záznamem typu A (IPv4 adresa) v zóně

Progtest ► BI-PA2 (25/26 LS) ► Domácí úloha 6 ► Domain Name System Logout

#1 v testovacím serveru, CNAME pro aliasy, ...). Pro jedno jméno lze zaznamenat více záznamů různého typu (např. lze zadat A a zároveň AAAA záznam, viz např. `www.fit.cvut.cz`), dokonce lze zadat více záznamů stejného typu s různým obsahem (například lze k jednomu jménu zaznamenat více záznamů typu A s různými adresami, klient náhodnou volbou adresy rozkládá zátěž, používá např. `google.cz`). Nepřípustné je pouze duplikování identických záznamů.

Reálné DNS je distribuované a redundantní. Naše implementace bude realizovat zjednodušení, které pomocí tříd realizuje ukládané záznamy a kontejnerový objekt, který tyto záznamy umožní vkládat, mazat a vyhledávat. Při implementaci se navíc omezíme pouze na záznamy A, AAAA a MX.

Úkolem je realizovat třídy CRecA, CRecAAAA a CRecMX, které budou reprezentovat záznamy v DNS. Dále je pak úkolem realizovat třídu CZone, která bude sloužit jako kontejner DNS záznamů. Při realizaci si můžete (a budete muset) vytvořit i další vlastní třídy. Dále budete používat třídy již připravené: CIPv4 a CIPv6 pro reprezentaci adres a standardní třídy pro reprezentaci výjimek. Připravené třídy jsou realizované v testovacím prostředí a v přiloženém archivu máte jejich základní implementaci, kterou budete potřebovat při ladění vašich tříd.

CRecA

Třída CRecA reprezentuje záznam typu A - překlad jména na IPv4 adresu. Rozhraní třídy musí obsahovat:

```
konstruktor
    připraví novou instanci záznamu ze zadaného jména a objektu IPv4 adresy.
name()
    metoda vrátí jméno ze záznamu.
type()
    metoda vrátí typ záznamu (zde A).
zobrazení ve streamu
    Záznam musí jít zobrazit do zadaného výstupního streamu (viz ukázka), bez odřádkování.
další
    do rozhraní můžete přidat další metody podle potřeby.
```

CRecAAAA

Třída CRecAAAA reprezentuje záznam typu AAAA - překlad jména na IPv6 adresu. Rozhraní třídy musí obsahovat:

```
konstruktor
    připraví novou instanci záznamu ze zadaného jména a objektu IPv6 adresy.
name()
    metoda vrátí jméno ze záznamu.
type()
    metoda vrátí typ záznamu (zde AAAA).
zobrazení ve streamu
    Záznam musí jít zobrazit do zadaného výstupního streamu (viz ukázka), bez odřádkování.
další
    do rozhraní můžete přidat další metody podle potřeby.
```

CRecMX

Třída CRecMX reprezentuje záznam typu MX - mail server pro zadanou doménu. Rozhraní třídy musí obsahovat:

```
konstruktor
    připraví novou instanci záznamu ze zadaného jména, jména mail serveru a jeho priority.
name()
    metoda vrátí jméno ze záznamu.
```

`type()`
metoda vrátí typ záznamu (zde MX).
zobrazení ve streamu
Záznam musí jít zobrazit do zadaného výstupního streamu (viz ukázka), bez odřádkování.
další
do rozhraní můžete přidat další metody podle potřeby.

CZone

Třída CZone reprezentuje zónu - kolekci DNS záznamů. Rozhraní musí umožňovat:

konstruktor
připraví novou prázdnou instanci pro zónu zadaného jména,
add()
metoda přidá záznam (A/AAAA/MX) do zadané zóny. Nový záznam je vždy přidáván na konec seznamu. Metoda vrátí `true` pro úspěch, `false` pro neúspěch (pokud by identický záznam v zóně již existoval).
del()
metoda smaže zadaný záznam. Kromě odstranění záznamu metoda nesmí měnit pořadí záznamů v zóně obsažených. Metoda vrátí `true` pro úspěch, `false` pro neúspěch (identický záznam v zóně neexistuje).
search()
metoda vyhledá v zóně záznam (záznamy) zadaného jména. S hodnotou, kterou metoda `search` vrátí, půjde provádět následující operace:

- zobrazení ve výstupním streamu - záznamy se zobrazí v podobě dle ukázek. Pořadí záznamů bude respektovat pořadí při vkládání,
- aplikováním metody `size` na výsledek půjde zjistit počet nalezených záznamů (`size_t`),
- aplikováním indexu půjde zpřístupnit jednotlivé nalezené záznamy, tedy na výsledek indexování půjde aplikovat metody `type`, `name` a výstupní operátor. Přetypováním hodnoty získané indexací pak půjde zpřístupnit i případné další metody z rozhraní `CRecA/CRecAAAA/CRecMX`. Použití je zřejmé z ukázek. Navíc, aplikace indexu mimo meze povede k vyhození výjimky `std::out_of_range`, v textu výjimky bude neplatný index v řetězcové podobě.

zobrazení ve streamu
zónu musí jít zobrazit do zadaného výstupního streamu, zobrazí se všechny obsažené záznamy (viz ukázka). Pořadí respektuje pořadí záznamů při vkládání.
kopírování
objekty zón se musí správně kopírovat/přiřazovat.
další
do rozhraní můžete přidat další metody podle potřeby.

Než začnete programovat věnujte dostatek času návrhu řešení. Zaměřte se na objektový návrh tak, abyste zbytečně neduplikovali kód. Správné řešení vyžaduje polymorfismus, testovací prostředí navíc zkoumá, zda jste třídy navrhli správně. Řešení bez polymorfismu nepůjde zkompileovat. Nepoužívejte RTTI (`typeid` nebude fungovat, úmyslně chybí hlavičkový soubor `typeinfo`), pokud možno nepoužívejte `dynamic_cast`. Pokud návrh podceníte, budete muset používat mnoho zbytečných větvení a výsledný program bude zbytečně dlouhý, nepřehledný a neudržitelný.

Odevzdávejte zdrojový kód s implementací požadovaných tříd (a dalších Vašich podpůrných tříd). Využijte přiložený archiv. Naleznete v něm soubor `test.cpp`, do kterého můžete umístit Vaši implementaci a hlavičkový soubor `ipaddress.h` s implementací pomocných tříd. Pokud implementaci požadovaných tříd umístíte do `test.cpp` a zachováte bloky podmíněného překladu, můžete tento soubor rovnou odevzdávat Progtestu.

Pro zvládnutí povinných testů postačí implementace používající naivní algoritmy. Ke zvládnutí bonusu je potřeba rychlé vyhledávání v zóně, kde je mnoho záznamů.

Poznámky:

- `dynamic_cast` lze při řešení použít, může se hodit např. pro porovnávání (tam jej používá i reference). Snažte se ale obejít bez `dynamic_cast` u přidávání a odebírání záznamů do CZone. Snahou je aby přidání dalšího typu záznamu nevynucovalo úpravy implementace CZone. Pokud je `dynamic_cast` použit v `CZone::add` nebo `CZone::del`, budete s každým přidaným typem záznamu muset tyto metody měnit. Pokud ale vhodně navrhnete rozhraní jednotlivých záznamů, budete používat `dynamic_cast` pouze uvnitř tříd implementujících tyto záznamy, navíc budete provádět `dynamic_cast` pouze na typ své třídy (tedy např. uvnitř implementace `CRecA` budete provádět `dynamic_cast<CRecA>`). Přidání nového typu záznamu tak nebude znamenat úpravy existujícího kódu.

Vzorová data:

Download

Odevzdat:

Vybrat soubor

Soubor nevybrán

Odevzdat

☐ Referenční řešení

2

18.04.2026 02:41:53

Download

Stav odevzdání:

Ohodnoceno

Hodnocení: 3.9600

• **Hodnotitel: automat**

- Program zkompileován
- Test 'Zakladni test podle ukazky': Úspěch
 - Dosaženo: 100.00 %, požadováno: 100.00 %
 - Celková doba běhu: 0.001 s (limit: 5.000 s)
 - Úspěch v závazném testu, hodnocení: 100.00 %
- Test 'Test navrhui trid': Úspěch
 - Dosaženo: 100.00 %, požadováno: 100.00 %
 - Celková doba běhu: 0.000 s (limit: 4.999 s)
 - Úspěch v závazném testu, hodnocení: 100.00 %
- Test 'Test nahodnymi hodnotami': Úspěch
 - Dosaženo: 100.00 %, požadováno: 50.00 %
 - Celková doba běhu: 0.129 s (limit: 4.999 s)
 - Úspěch v závazném testu, hodnocení: 100.00 %
- Test 'Test kopirujiciho konstruktora': Úspěch
 - Dosaženo: 100.00 %, požadováno: 50.00 %
 - Celková doba běhu: 0.013 s (limit: 4.870 s)
 - Úspěch v závazném testu, hodnocení: 100.00 %
- Test 'Test operatoru=': Úspěch
 - Dosaženo: 100.00 %, požadováno: 50.00 %
 - Celková doba běhu: 0.015 s (limit: 4.857 s)
 - Úspěch v závazném testu, hodnocení: 100.00 %
- Test 'Test nahodnymi daty + mem dbg': Úspěch
 - Dosaženo: 100.00 %, požadováno: 50.00 %
 - Celková doba běhu: 0.003 s (limit: 5.000 s)
 - Úspěch v závazném testu, hodnocení: 100.00 %
- Test 'Test rychlosti': Úspěch
 - Dosaženo: 100.00 %, požadováno: 100.00 %
 - Celková doba běhu: 0.607 s (limit: 5.000 s)
 - Úspěch v bonusovém testu, hodnocení: 120.00 %
- Celkové hodnocení: 120.00 % (= 1.00 * 1.00 * 1.00 * 1.00 * 1.00 * 1.00 * 1.20)
- Celkové procentní hodnocení: 120.00 %
- Bonus za včasné odevzdání: 0.30
- Celkem bodů: $1.20 * (3.00 + 0.30) = 3.96$

SW metriky:

1	18.04.2026 02:25:23	Download
Stav odevzdání:	Ohodnoceno	
Hodnocení:	3.3000	
• Hodnotitel: automat ◦ Program zkompileován ◦ Test 'Zakladni test podle ukazky': Úspěch ▪ Dosaženo: 100.00 %, požadováno: 100.00 % ▪ Celková doba běhu: 0.000 s (limit: 5.000 s) ▪ Úspěch v závazném testu, hodnocení: 100.00 % ◦ Test 'Test navrhui trid': Úspěch ▪ Dosaženo: 100.00 %, požadováno: 100.00 % ▪ Celková doba běhu: 0.000 s (limit: 5.000 s) ▪ Úspěch v závazném testu, hodnocení: 100.00 % ◦ Test 'Test nahodnymi hodnotami': Úspěch ▪ Dosaženo: 100.00 %, požadováno: 50.00 % ▪ Celková doba běhu: 0.323 s (limit: 5.000 s) ▪ Úspěch v závazném testu, hodnocení: 100.00 % ◦ Test 'Test kopirujiciho konstruktora': Úspěch ▪ Dosaženo: 100.00 %, požadováno: 50.00 % ▪ Celková doba běhu: 0.105 s (limit: 4.677 s) ▪ Úspěch v závazném testu, hodnocení: 100.00 % ◦ Test 'Test operatoru=: Úspěch ▪ Dosaženo: 100.00 %, požadováno: 50.00 % ▪ Celková doba běhu: 0.129 s (limit: 4.572 s) ▪ Úspěch v závazném testu, hodnocení: 100.00 % ◦ Test 'Test nahodnymi daty + mem dbg': Úspěch ▪ Dosaženo: 100.00 %, požadováno: 50.00 % ▪ Celková doba běhu: 0.003 s (limit: 5.000 s) ▪ Úspěch v závazném testu, hodnocení: 100.00 % ◦ Test 'Test rychlosti': Program překročil přidělenou maximální dobu běhu ▪ Vyčerpání limitu na celý test, program násilně ukončen po: 5.005 s (limit: 5.000 s)		

- Neúspěch v bonusovém testu, hodnocení: Bonus nebude udělen
 - Celkové hodnocení: 100.00 % ($= 1.00 * 1.00 * 1.00 * 1.00 * 1.00 * 1.00$)
- Celkové procentní hodnocení: 100.00 %
- Bonus za včasné odevzdání: 0.30
- Celkem bodů: $1.00 * (3.00 + 0.30) = 3.30$

SW metriky: